Course project: Frames in C++ *

Programmation Objet 1

E4 IMAC, 2026-2027



Instructions

- This project can be submitted in groups of m students with $m \in \{1, 2\}$. Upon submission, the project name should clearly indicate the names of all students involved.
- Students should submit sources of their code (`.cpp`, `.hpp` files). They should either include explicit compiling instructions or a `Cmake` (or both). Including executable files is not necessary.
- The deadline for submitting the project on e-learning is **November 29 AOE**.
- Students are allowed to use AI tools to assist them during the project realization. However, students **are forbidden to** upload the assignment (in PDF or any other format) directly into an AI tool.
- Parts of the assignment are unspecified on purpose, as there are usually several valid ways of implementing the same function or class. If students believe that an additional function or method would be useful for their project, they are more than welcome to implement it.
- For any typo, question, etc, please post a message on e-learning or send an email to `clement.royer@univ-eiffel.fr`.

History of the document

- 2026.10.29: First version.

*Current version: October 29, 2026.

Background: Frames

The goal of this project is to represent frames in C++. Frames are specific sets of vectors that have numerous applications in signal processing and optimization. They can be used to approximate functions (like sines and cosines in Fourier analysis) or vectors in finite dimension. In this project, we view frames as sets of vectors in $\mathbb{R}^n$ (where n is a positive integer), where a vector will be understood as an `std::vector`.

More precisely, a *frame* $\mathcal{F}$ is a family of nonzero vectors of $\mathbb{R}^n$ (possibly with duplicates). For every nonzero vector $v \in \mathbb{R}^n$, the *quality of* $\mathcal{F}$ *at* v is defined by the mathematical quantity

$$q(\mathcal{F}, v) = \max_{d \in \mathcal{F}} \frac{\langle d, v \rangle}{\|d\| \|v\|}, \tag{1}$$

where $\langle d, v \rangle = \sum_{i=1}^{n} d_i \, v_i$ is the inner product on $\mathbb{R}^n$ and $\|u\| = \sqrt{\langle u, u \rangle}$ is the corresponding norm. In general, we have $q(\mathcal{F}, v) \in [-1, 1]$, and we consider that the frame $\mathcal{F}$ gives a good approximation of the vector v as long as $q(\mathcal{F}, v) > 0$. If $q(\mathcal{F}_1, v) > q(\mathcal{F}_2, v)$ for two frames $\mathcal{F}_1$ and $\mathcal{F}_2$, we say that $\mathcal{F}_1$ gives a better approximation of v than $\mathcal{F}_2$.

In general, frames with more elements have higher quality measure, which is why we also consider the *normalized quality measure* defined by

$$q_N(\mathcal{F}, v) = \frac{q(\mathcal{F}, v)}{|\mathcal{F}|}, \tag{2}$$

where $|\mathcal{F}|$ represents the number of elements within a frame, or cardinality. Computing the best frame of fixed cardinality has strong ties to the Tammes problem[1], which is still open in many cases as of the day this assignment is written.

[1] Sometimes called *problème des dictateurs* in French, see https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_Tammes.

Part 1: A basic class

a) Implement a class called `Frame3D` that encodes frames formed by vectors of 3 `double` coefficients. *Use the structure of your choice to store the family of vectors.*

b) Add a copy constructor, a destructor and an overloaded affectation operator.

c) Write a method `size` that outputs the number of vectors contained in a frame.

d) Write a method `add` that adds a vector (passed as an `std::vector`) to an existing frame:

```
void add(std::vector<double> & v);
```

e) Overload the `+` operator to sum two frames into a new one representing the union of both frames. For instance, if $\mathcal{F}_1 = \{d_1, d_2\}$ and $\mathcal{F}_2 = \{d_1, d_3, d_4\}$, then the frame $\mathcal{F}_1 + \mathcal{F}_2$ should consist of $\{d_1, d_1, d_2, d_3, d_4\}$.

f) Write a method `quality_measure` that computes the quantity (1) for a frame $\mathcal{F}$ and a vector v, passed as a `std::vector`.

```
double quality_measure(const std::vector<double> & v);
```

g) Add assertions to your code to guarantee that only nonzero vectors are used.

h) **Application** Consider the following frames:

$$\begin{aligned}
\mathcal{F}_1 &= \left\{ \begin{bmatrix}1\\0\\0\end{bmatrix}, \begin{bmatrix}-1\\0\\0\end{bmatrix}, \begin{bmatrix}0\\1\\0\end{bmatrix}, \begin{bmatrix}0\\-1\\0\end{bmatrix}, \begin{bmatrix}0\\0\\1\end{bmatrix}, \begin{bmatrix}0\\0\\-1\end{bmatrix} \right\}, \\
\mathcal{F}_2 &= \left\{ \begin{bmatrix}1\\0\\0\end{bmatrix}, \begin{bmatrix}0\\1\\0\end{bmatrix}, \begin{bmatrix}0\\0\\1\end{bmatrix}, \begin{bmatrix}-\frac{1}{\sqrt{3}}\\-\frac{1}{\sqrt{3}}\\-\frac{1}{\sqrt{3}}\end{bmatrix} \right\}, \\
\mathcal{F}_3 &= \mathcal{F}_1 + \mathcal{F}_2.
\end{aligned}$$

Compute their quality measure at the points $\begin{bmatrix}\frac{1}{\sqrt{3}}\\\frac{1}{\sqrt{3}}\\\frac{1}{\sqrt{3}}\end{bmatrix}$ and $\begin{bmatrix}1-\frac{1}{\sqrt{3}}\\-\frac{1}{\sqrt{3}}\\-\frac{1}{\sqrt{3}}\end{bmatrix}$. Which frame provides the best approximation in terms of quality measure? Is the answer different if we use the normalized quality measure?

Part 2: A template for frames

a) Create a template

```
template <typename T,int n> class Frame;
```

that represents abstract frames. The expression parameter n is the dimension of the vectors.

b) Implement the template so that all methods implemented in the `Frame3D` class have an equivalent in the `Frame` class template.

c) Redefine the `quality_measure` function for the `bool` type so that the result outputs (still as a `double`) the quantity

$$\max_{d\in\mathcal{F}} \frac{|I_d \cap I_v|}{|I_d \cup I_v|},$$

where $\mathcal{F}$ represents the `Frame` object, v is the input of `quality_measure`, and for any vector u of n booleans, I_u is the set of all indices for which $u_i =$`true`.[2]

d) Create an exception class `AddFrameException` that is thrown when the function `add` is called using a vector of a different length than that used by the frame, and catch that exception by printing a message and interrupting the program.

e) Create a class template `MinimalFrame` that inherits from `Frame` and does not contain multiple copies of the same element. This class template should have an additional constructor that creates a `MinimalFrame` from a `Frame`:

```
MinimalFrame(const Frame &);
```

f) **Application** Create minimal frames from the frames $\mathcal{F}_1$, $\mathcal{F}_2$, $\mathcal{F}_3$ introduced in the first part. Compare the resulting quality measure and normalized quality measure on the examples from the first part. Are the results different when the frames are encoded using `float` or `double` vectors?

g) **Application** Consider the boolean frames

$$\mathcal{F}_4 = \left\{ \begin{bmatrix} \texttt{true} \\ \texttt{false} \\ \texttt{false} \\ \texttt{false} \end{bmatrix}, \begin{bmatrix} \texttt{false} \\ \texttt{true} \\ \texttt{false} \\ \texttt{false} \end{bmatrix}, \begin{bmatrix} \texttt{false} \\ \texttt{false} \\ \texttt{true} \\ \texttt{false} \end{bmatrix}, \begin{bmatrix} \texttt{false} \\ \texttt{false} \\ \texttt{false} \\ \texttt{true} \end{bmatrix} \right\}$$

$$\mathcal{F}_5 = \left\{ \begin{bmatrix} \texttt{true} \\ \texttt{false} \\ \texttt{true} \\ \texttt{false} \end{bmatrix}, \begin{bmatrix} \texttt{false} \\ \texttt{true} \\ \texttt{false} \\ \texttt{true} \end{bmatrix} \right\}.$$

Which frame provides the best approximation of the vector $\begin{bmatrix} \texttt{true} \\ \texttt{true} \\ \texttt{false} \\ \texttt{false} \end{bmatrix}$ in terms of quality measure? Is the answer different if we use the normalized quality measure?

[2] For instance, if `u=[true,false,true]`, and `v=[true,true,false]` then $I_u = \{0, 2\}$ (with the convention that indexing starts with 0), $I_v = \{0, 1\}$, $I_u \cap I_v = \{0\}$ and $I_u \cup I_v = \{0, 1, 2\}$.

Part 3: A special type of integer frames

A vector of n integer values is called *primitive* if the greatest common divisor (gcd) of its coefficients is 1.[3] In C++, the gcd is computed using the `std::gcd` routine from the `numeric` library since C++17.

The goal of this part of the project is to build primitive frames, i.e. frames that consist only of primitive vectors.

a) Implement a function

```
void turn_primitive(std::vector<int> &);
```

that transforms a vector of integers into a primitive one.

b) Create a class template `PrimitiveFrame` that inherits from `Frame<int,n>` and has an additional attribute `val_max` indicating the largest possible (absolute) value of the vectors' coordinates[4].

c) Overload or redefine methods as necessary to guarantee that a `PrimitiveFrame` only contains primitive vectors. Make sure, however, that the `add` function can still receive any vector as an input (not just a primitive vector).

d) Create an exception class `AddPrimitiveFrameException` that handles calls to `add` with a vector whose components are too large for the frame (in the sense of `val_max`).

e) **Application** Build the largest possible `PrimitiveFrame` using $n = 2$ and `val_max=3`. Compute its quality measure with respect to the vector $\begin{bmatrix} -2 \\ 2 \end{bmatrix}$ and the corresponding normalized quality measure. Compare these values with that obtained for the `PrimitiveFrame` defined by

$$\mathcal{F}_6 = \left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ -1 \end{bmatrix} \right\}.$$

[3] In French, the greatest common divisor is called the PGCD (*Plus Grand Commun Diviseur*). For instance, the gcd of $(1, 2, 4)$ is 4, while that of $(3, 34, 12)$ is 1.

[4] For instance, if that value is 1, then the frame should consist of vectors with coordinates in $\{-1, 0, 1\}$.